

QUELLCODEKRITIK. README1ST.TXT

Hannes Bajohr und Markus Krajewski

Algorithmen, so könnte man in Anlehnung an einen Satz Friedrich Kittlers sagen, bestimmen unsere Lage. Von *bubble sort* und Googles PageRank über Kreditwürdigkeitsprüfungen bis hin zum *predictive policing* – die Logik der Algorithmen greift auf Schritt und Tritt in unser Leben ein. Einige arbeiten undurchsichtig und sind gut gehütete Geschäftsgeheimnisse. Bei anderen bemüht man sich um Transparenz und folgt einer Ethik des *open source* und der Rechenschaftspflicht. In allen Fällen ist jedoch ein nicht unerheblicher Aufwand erforderlich, um die Quellcodes zu verstehen, in denen Algorithmen normalerweise niedergelegt sind. Das ist auch eine Herausforderung für die Geisteswissenschaften, von denen nur ein kleiner Teil über die nötige, wie Annette Vee es nennt, *coding literacy* verfügt, um Algorithmen mit Verständnis zu lesen und zu interpretieren, zu kommentieren und weiterzuschreiben.¹

Die Gründe, aus denen Code kein unproblematischer Gegenstand klassischer Lektüre ist, sind zahlreich. In den allermeisten Fällen ist er seinen potentiellen Leser:innen schlicht unzugänglich. Sie stehen, wenn sie ein Programm öffnen, nicht mehr dem Quellcode gegenüber, sondern einer ausführbaren Datei, die im Binärformat vorliegt. Dieser Maschinencode ist für Menschen praktisch nicht zu durchschauen. Zwar kann ein solches kompiliertes Programm über ein sogenanntes *reverse engineering* in eine Assemblersprache (bei JAVA sogar in den vollständigen ursprünglichen Quelltext ohne Kommentare) zurückübersetzt werden. Aber eine solche Transformation ist höchstens Code *a posteriori*, ein nachträgliches Produkt. Dessen

¹ Annette Vee, *Coding Literacy. How Computer Programming is Changing Writing*, Cambridge, MA: MIT Press 2017.

bloße Auflistung der allerbasalsten Kommandos und Datenstrukturen ist daher zumeist weit entfernt vom ursprünglichen Quellcode, der in den meisten Fällen in einer höheren Programmiersprache wie C, C++ oder Python geschrieben ist.

Immer öfter aber ist das, was man den Code *a priori* nennen könnte, doch frei zugänglich. Sei es, weil Entwickler:innen ältere Programme samt Quellcode in die Öffentlichkeit entlassen, sei es, weil sie bereits von vornherein unter verschiedenen Open-Source-Lizenzen über reichhaltige Online-Repositorien wie etwa Microsofts github.com weitergegeben werden. Aber auch dieser frei verfügbare Code *a priori* ist keinesfalls trivial zu lesen. Denn auch Quellcode, der nicht hinter Barrieren versteckt ist oder bewusst unlesbar gemacht wurde, widersetzt sich Lesenden, die keine technische Expertise zu seiner Lektüre mitbringen – sei diese in einem Informatikstudium erworben oder auch nur bloßes Amateurwissen. Dabei ist es heute dringender denn je, auch außerhalb dieser Gruppen zeitgemäße Medienkompetenz in Form zuverlässiger *coding literacy* zu fördern.

Auch in den Geistes-, Sozial- und Rechtswissenschaften sollten Wissenschaftler:innen die Fähigkeiten erwerben und weitergeben, die notwendig sind, um nicht nur komplexe philosophische, juristische und literarische Sachverhalte in Textform zu erforschen, sondern eben auch Codes zu verstehen und sie aus einer kritischen Perspektive einordnen zu können. Denn obwohl Code eine Art von Sprache ist und von Menschen und Maschinen gleichermaßen geschrieben und gelesen wird, vermuten die philologischen Fakultäten Algorithmen zumeist außerhalb ihres Fachbereichs und ihrer Kompetenzen. Selbst die Medienwissenschaft, unsere eigene Disziplin, die über die Instrumente und Konzepte verfügen sollte, um sich mit ihnen zu befassen, bleibt oft zurückhaltend und zieht es vor, nur abstrakt über algorithmische Macht zu sprechen.

Die Behauptung, Algorithmen seien abstrakte Artefakte, ist an sich nicht falsch. Aus kulturhistorischer Sicht bezeichnen

Algorithmen eine Rechenregel, ein Kalkül, das aus einer linearen Abfolge von Befehlen besteht. Von einem klar definierten Startpunkt aus wird er Schritt für Schritt ausgeführt, um eine bestimmte Aufgabe oder ein Problem zu lösen. Ein Algorithmus kann daher als eine in sich geschlossene Kette von Kommandos oder als eine Operationskette betrachtet werden. Ist diese Kette in einer Programmiersprache geschrieben, spricht man von Code (wobei dieser neben Algorithmen etwa auch Datenstrukturen, Bibliotheken und andere Ressourcen umfassen kann).

Um wirksam zu sein, benötigt Code einen Computer, der ihn ausführt. Wohl keine Technik kann nur für sich allein funktionieren, ohne die Mitwirkung anderer. Gerade deshalb ist es naheliegend zu sagen, dass Algorithmen nicht allein abstrakte Gebilde sind, sondern auch eine soziale Wirklichkeit besitzen, weil ihre Funktionen immer an den Schnittstellen zwischen Maschinen und Menschen angesiedelt sind: Sie verarbeiten nicht nur datentechnisch menschliche Eingaben, sondern in den allermeisten Fällen entwerfen Softwareentwickler:innen – im Moment noch überwiegend Menschen – die konkreten algorithmischen Abläufe. Gleichzeitig modellieren die praktisch umgesetzten Algorithmen ihre Nutzer:innen, beispielsweise wenn diese beim Gebrauch einer bekannten Suchmaschine die *cookies* akzeptieren und infolgedessen mithilfe vieler Messpunkte individuelle Eigenschaften wie Computerkonfiguration, Browser, Betriebssystem, Sprache etc. erfasst werden.

Darüber hinaus könnten Algorithmen aber auch in einem anthropologischen Sinne als Operationsketten, als *chaîne opératoire*,² beschrieben werden. In diesen Operationsketten werden verschiedene Akteur:innen – menschlicher wie technischer Art – zu einer temporären Handlungseinheit zusam-

² André Leroi-Gourhan, *Hand und Wort. Die Evolution von Technik, Sprache und Kunst*, Frankfurt a.M.: Suhrkamp 1965/1980, S. 150f. und S. 275–280.

mengefasst, die erst durch das Zusammenspiel von Objekten, Medien und Personen eine bestimmte Wirkung erzielt. Eine Operationskette organisiert das Zusammenspiel einer manuellen Geste (Technik), eines Werkzeugs (*l'outil*) und symbolischer Operationen (Sprache), um eine Handlung zu vollziehen. Operationsketten sind daher weder nur Technik und Prozess, sondern eben auch soziale Konstrukte mit sozialen Auswirkungen, weil sie die Interaktion zwischen menschlichen und nicht-menschlichen Akteuren konstituieren und begleiten.

Diese Charakterisierung versteht Algorithmen als soziale Akteure. Als Handlungskette kann die Ausführung eines Algorithmus nicht nur virtuelle und soziale, sondern auch physisch-materielle Wirkungen erzeugen. In einer solchen anthropologischen Perspektive verwandeln Algorithmen als Operationsketten bestimmte Ideen in konkrete Handlungen, indem sie spezifische Werkzeuge (z.B. Filter) oder Techniken (in der Hardware ebenso wie in der Software) verwenden.

Code handelt demnach als Akteur in komplexen Operationsketten, und seine Auswirkungen sind nicht auf den Computer allein beschränkt, sondern spielen in gesellschaftliche, rechtliche und wirtschaftliche Beziehungen in der realen Welt hinein. Derart situierte Algorithmen sind, anders gesagt, in soziale Beziehungen eingebettet und agieren als deren Konstrukte an den Schnittstellen zwischen Mensch und Maschine, werden in konkreten Plattformen implementiert und von realen Menschen geschrieben. Als Grenzobjekte, deren Effekte sich auf eine Vielzahl von sozialen Feldern auswirken, kann ihr Tun verfolgt und ihre Interaktion mit der Realität bewertet werden – ohne jemals *in* diese Objekte hineinzuschauen.

Aber auch das ist noch eine Art Vogelperspektive, die nur ein Teil einer wirklich umfassenden Codephilologie sein kann. Eine solche Makrokomplexität kann nur dann sinnvoll analysiert werden, wenn die Funktionsweise ihrer Objekte auch auf der *Mikroebene* verstanden wird. Mit anderen Worten: Es ist

erforderlich, Code auf den Grund zu gehen und ihn tatsächlich zu lesen. Das heißt, die Texte von höheren Programmiersprachen einer Lektüre zu unterziehen.

Dass eine solche Lektüre gewisse Probleme mit sich bringt, die über den Rahmen der klassischen Hermeneutik hinausgehen, ist offensichtlich, und David Berry, Mark C. Marino oder Kevin Brock haben dies hinreichend deutlich gemacht.³ Codes sind schließlich ganz besondere Sprachartefakte. Sie führen Aktionen aus, wenn sie ausgeführt werden, was Florian Cramer dazu veranlasste, sie in einer Linie mit Zaubersprüchen und anderen „selbstaussführenden Texten“ zu sehen.⁴ Mit einer metonymischen Charakteristik Kittlers gesprochen erhält die Schrift damit ein ganz eigenes, neues Handlungsvermögen: „Die ENTER-Taste hat eine Macht erlangt, die den Wortsinn von Poesie, nämlich ‚machen‘, im Unterschied zu aller Poesie oder Literatur der Geschichte erstmals einlöst.“⁵

Potenziert der Code die pragmatische Seite der Sprache (ohne im wirklichen Sinne Sprechakt zu sein),⁶ reduziert er seine expressive Dimension auf die reine Denotation. Er ist, wie Charles Taylor es nannte, „depiction without expression“⁷ – ein Status, den keine sogenannte „natürliche Sprache“ erreichen kann, auch wenn Code für gewöhnlich einen nicht geringen Anteil an natürlicher Sprache beinhalten muss, um lesbar zu bleiben. Weil er zunächst keinerlei konnotative oder referenzielle Dimension hat, treibt er die Konventionalität von

³ David M. Berry, *The Philosophy of Software. Code and Mediation in the Digital Age*, Basingstoke: Palgrave Macmillan 2011; Kevin M. Brock, *Rhetorical Code Studies: Discovering Arguments in and around Code*, Ann Arbor: University of Michigan Press 2019; Mark C. Marino, *Critical Code Studies*, Cambridge, MA: MIT Press 2020.

⁴ Florian Cramer, *Exe.cut[up]able statements. Poetische Kalküle und Phantasmen des selbstaussführenden Texts*, München: Fink 2011.

⁵ Friedrich Kittler, *Daten → Zahlen → Codes. Vortrag an der Hochschule für Grafik und Buchkunst*, Leipzig: Institut für Buchkunst 1998, S. 20.

⁶ Vgl. Martin Bartelmus und Alexander Nebrig (Hg.), *Schriftlichkeit. Aktivität, Agentialität und Aktanten der Schrift*, Bielefeld: Transcript 2022.

⁷ Charles Taylor, „Theories of Meaning“, in: ders., *Human Agency and Language. Philosophical Papers I*, Cambridge: Cambridge University Press 1985, S. 247–292, hier S. 267.

Sprache ins Extrem: Alle Ähnlichkeiten zu natürlichsprachigen Wörtern sind in Programmiersprachen reine Bequemlichkeit zum Erlernen für menschliche User:innen. Damit sind sie zwar auf Lesbarkeit ausgelegt, die ist aber von der Regel- und Anweisungshaftigkeit ihrer Ausführung abgekoppelt – Lektüre- und Ausführungsfunktion klaffen auseinander. Der Programmierpapst Donald Knuth soll daher gesagt haben: „Programmieren ist die Kunst, einem anderen Menschen zu sagen, was der Computer tun soll.“⁸

Und schließlich wird Code auf ansonsten träge Datenstrukturen angewandt. Damit steht eine Philologie von Quellcodes vor Text, der auf einen anderen Text einwirkt und damit eine doppelte, wenn nicht dreifache Komplexität aufwirft: Nicht nur stehen menschliche in Konkurrenz zu nichtmenschlichen Lesenden, wie Compilern oder Datenübertragungsprotokollen; der Text steht auch in Konkurrenz zu und in Interaktion mit sekundärem Text, auf den er zugreifen muss, um funktionsfähig zu sein; und er muss funktionsfähig sein, um für Leser:innen zugänglich zu sein. Kurz: In Code niedergelegte Algorithmen sind also gleichzeitig *mehr* und *weniger* als natürliche Sprache. Und sie erfordern eine Philologie, die in der Informatik ebenso zu Hause ist wie in der Textkritik.

Der vorliegende Band geht von dieser Annahme aus. Er verzamelt Beiträge einer Reihe von Wissenschaftler:innen aus verschiedenen Disziplinen mit je eigenen Perspektiven auf Algorithmen und ihre Lesbarkeit, um drei Fragen zu diskutieren: 1) Wie kann Code verstanden werden? 2) Wie kann er verwaltet und gepflegt werden? 3) Und wie kann man dazu beitragen, die in weiten Teilen der Geisteswissenschaften immer noch fehlende Codekompetenz zu fördern? Oder anders

⁸ Ein unbelegter Aphorismus, der immer wieder zitiert wird, etwa in: Selma Takir, „Reading CS Classics“, in: *Communications of the ACM* 55:4 (2012), S. 32–34, hier S. 33. Mark C. Marino zitiert in seinem Aufsatz einen sinnähnlichen, aber weniger eingängigen Ausspruch, siehe S. 35.

gesagt: Es geht um die Hermeneutik, die Philologie und die Didaktik von Algorithmen.

Eine *Hermeneutik* von Algorithmen muss, wie betont, die soziale Realität von Quellcodes ebenso berücksichtigen wie ihre textuelle Mikrologie; sie vermischt die interne Bedeutung eines Codes – den Verweis auf seine eigenen Strukturen, Traditionen und kanonischen Regelmäßigkeiten – mit der Macht eines jeden Textes, stets über sich selbst hinauszudeuten und sich in das soziale Leben von Individuen und Gruppen, die Aktivitäten von Volkswirtschaften und die Regeln und Regelbefolgung von Staaten, juristischen Systemen und staatsähnlichen Akteur:innen einzufügen. Eine solche Hermeneutik muss eine Vorstellung vom ontologischen Status der Algorithmen sowie von ihrer sozialen, rechtlichen und wirtschaftlichen Handlungsfähigkeit haben. Sie sollte auch ein kritisches Auge auf die Frage werfen, in welchen Konstellationen ihre scheinbare Objektivität im Widerspruch zu ihrer kulturellen Einbettung steht.

Eine *Philologie* des Codes ist mehr als das. Sie macht den Code selbstreflexiv und fügt dem Text die Werkzeuge der philologischen Kritik und Pflege hinzu; sie folgt der These, dass der Quellcode so intensiv wie möglich dokumentiert werden sollte, damit die Algorithmen selbsterklärend werden. Schließlich lädt uns das performative Element des Quellcodes im Gegensatz zu einem gewöhnlichen Text auch dazu ein, in ihn einzugreifen und seinen Umfang, seine Möglichkeiten und seinen Nutzen zu erkunden und zu verändern. Textkritik und Hermeneutik werden so praktisch und interaktiv.

Hierin liegt auch im weitesten Sinne eine *Didaktik* von Code. Denn eine solche Interaktion ist eine der Hauptaufgaben der gegenwärtigen Geisteswissenschaften – eine Vertrautheit mit Programmierung als solcher zu entwickeln, die die Kluft zwischen der Informatik und den interpretierenden Disziplinen überbrückt – eine Forderung, die von programmierenden

Geisteswissenschaftler:innen seit Längerem, jedoch ohne größere Resonanz erhoben wurde.⁹ Computeralphabetismus gilt es also nach wie vor – zumindest als passive Fähigkeit – in den Geisteswissenschaften weiter zu verbreiten: Es ist eine kritische Fähigkeit, die notwendig ist, um Code zu verstehen, offenzulegen, einzuordnen, zu kontextualisieren und zu erklären, um der viel beschworenen Macht der Algorithmen zu begegnen. Um zu erkennen, wie genau Benutzer:innen von einer Software benutzt werden, ist es notwendig, das Design und die Konstruktion eines Algorithmus zu entschlüsseln, zu verstehen und kritisch aufzudecken.

Einer solchen Quellcodekritik geht es nicht zuletzt darum, dass Code Geschichte hat. Aber diese Geschichte ist noch nicht an ihrem Ende. In den vergangenen Jahren nämlich hat sich das Wesen der Programmierung verändert. Der klassische Code ist heute nicht mehr die einzige Option für die Entwicklung komplexer oder auch einfacher Applikationen. Das klassische *sequenzielle* Paradigma mit seinen Regelschritten, Verzweigungen und Loops wird heute durch das *konnektionistische* Paradigma¹⁰ – stochastische maschinelle Lernmodelle – zumindest erweitert, wenn nicht infrage gestellt. Solche KI-Modelle bestehen aus so etwas *wie* Algorithmen – wenn man unter ‚Algorithmus‘ eine Funktion versteht und darunter die Abbildung von einer Menge von Eingaben auf eine Menge von Ausgaben –,¹¹ sind aber nicht mehr in derselben Weise *lesbar* wie

⁹ Vgl. Friedrich Kittler, „Den Riß zwischen Lesen und Schreiben überwinden. Im Computerzeitalter stehen die Geisteswissenschaften unter Reformdruck“, in: *Frankfurter Rundschau*, 12. Januar 1993, S. 16, sowie ders., „Computeranalphabetismus“, in: Dirk Matejovski und Friedrich Kittler (Hg.), *Literatur im Informationszeitalter*, Schriftenreihe des Wissenschaftszentrums Nordrhein-Westfalen, Bd. 2, Frankfurt a.M.: Campus 1995, S. 237–251.

¹⁰ Vgl. zu dieser Unterscheidung: Hannes Bajohr, „Algorithmische Einfühlung. Für eine Kritik ästhetischer KI“, in: ders., *Schreibenlassen. Texte zur Literatur im Digitalen*, Berlin: August 2022, S. 131–172.

¹¹ Vgl. Juan Luis Gastaldi, „The Place of Textuality in Deep Learning Approaches to Mathematics“, 2023, https://www.giannigastaldi.com/assets/pdf/pubs/Gastaldi2023b_s.pdf [Letzter Zugriff: 15.8.2023].

beispielsweise ein Python-Code. Modelle für maschinelles Lernen sind, wie man oft sagt, *Blackboxes*.¹²

Damit meint man in der Regel, dass insbesondere die Gewichtungsmodelle (*weight models*) neuronaler Netze oder anderer Arten von maschinellen Lernsystemen in ihrer Funktionsweise relativ undurchsichtig sind. Ein Standardeinwand dagegen lautet, dass das Forschungsfeld der sogenannten *explainable AI* (oder *XAI*) bereits eine Vielzahl von Imaging-Methoden aufgeboten hat, die helfen, das Innere dieser Modelle für menschliches Verständnis zu öffnen. Keines dieser Verfahren ist jedoch in der Lage, das Innenleben eines stochastischen Modells in die Regelschritte eines sequenziellen Algorithmus zu übersetzen, der schließlich wieder menschenlesbar wäre. Und es ist nicht unwahrscheinlich, dass dies im Prinzip unmöglich ist. Der Punkt ist also nicht, dass die Blackbox völlig dunkel ist, sondern dass kein noch so grelles Licht uns dieselbe Klarheit verschaffen kann, die ein sequenzieller Algorithmus ermöglicht.

Dies ruft einige der grundlegendsten Annahmen der Hermeneutik und der Philologie gleichermaßen auf: dass sie, um August Boeckh zu zitieren, „die Erkenntnis des Erkannten“¹³ seien – und sich als solche mit menschlichen kulturellen Artefakten befassen, deren Verständnis eine Rückverfolgung der Praktiken und Kenntnisse ist, die in sie eingeflossen sind; das trennt sie von den Naturwissenschaften, deren Aufgabe nur eine funktionale Erklärung, aber niemals ein Verstehen sein kann. Diese Unterscheidung kann auf den Unterschied zwischen dem klassischen, sequenziellen Code und den neuen, konnektionistischen Modellen des maschinellen Lernens

¹² Vgl. Markus Krajewski, „Source Code Criticism. On Programming as a Cultural Technique and its Judicial Linkages“, in: Laurence Diver und Mireille Hildebrandt (Hg.), *The Legal Effect of Code-Driven Law. Journal of Cross-Disciplinary Research in Computational Law* 2024 (in Vorbereitung).

¹³ August Boeckh, *Enzyklopädie und Methodenlehre der philologischen Wissenschaften*, hg. v. Ernst Bratuscheck, Leipzig: Teubner 1877, S. 18, Orthografie angepasst.

übertragen werden: Bei dem einen geht es um Verstehen, bei dem anderen um Erklären. Neuronale Netze als Forschungsobjekte sind durch eine Außenperspektive bestimmt – sie werden nicht anders untersucht als Gehirnstrukturen oder Sterncluster; der traditionelle Code hingegen lädt zu einer hermeneutischen, zu einer Innenperspektive ein und ist auf Verstehen ausgerichtet. *Verum ipsum factum*, der Grundsatz, der den Unterschied zwischen Naturalismus und Hermeneutik ausmacht, gilt hier: Ein von Menschen geschriebener Code wird immer anders gelesen werden als ein maschinell erstelltes Gewichtungsmodell – ungeachtet aller bildgebenden Verfahren der *explainable AI*.

Bemerkenswert bleibt dabei jedoch, dass die Herstellung der Blackboxes, die Produktion ihrer prinzipiellen Undurchschaubarkeit wiederum durch sequenziellen Code angeleitet oder programmiert wird. Keine Blackbox, etwa ein großes Sprachmodell (*large language model*, LLM), wie es ChatGPT zugrunde liegt, kann ohne das Skript in einer „klassischen“ Programmiersprache erstellt werden. Auf dieser übergeordneten Ebene fusionieren die beiden Paradigmen in einer solcher Weise, dass sich erst hier die Bedingungen des zu Erklärenden einsehen und nachvollziehen lassen. Es ist auf dieser Ebene der Skripte, der Rezepturen gleichsam, wo sich die Ausgangslage zeigt, also etwa die herangezogenen Korpora eines LLMs, der Fundus der Bilder, auf dem die Version eines Bildgenerators wie *midjourney* trainiert wurde, oder jene Teile des Internets, die in ihrer Bandbreite zwischen mäßiger Qualität und hervorragender Strukturiertheit zur Ausbildung von Weltwissen einer solchen Blackbox eingebunden wurden. Erst durch eine explizite Listung dieser Quellen im Quellcode und die Offenlegung dieser Referenzen lässt sich das Training eines Modells einigermaßen transparent und damit kritisierbar machen – auch wenn sich das Modell intern mit seiner Versammlung von Milliarden

numerischer Vektoren weiterhin einer vollen und direkten Lesbarkeit entzieht.¹⁴

Gemäß der Unterscheidung der beiden Paradigmen von sequenziellen und konnektionistischen Codes haben wir den Band strukturiert. Der erste Teil beschäftigt sich mit klassischen Befehlsfolgen, mit sequenziellen Algorithmen, der zweite diskutiert die Herausforderungen des konnektionistischen Paradigmas im maschinellen Lernen. Diese Unterscheidung, im Rahmen eines breiteren Codealphabetismus verstanden, geht dabei auch weit über das hinaus, was die *Critical Code Studies* traditionell verfolgt haben.

Den Auftakt macht Mark C. Marino mit seinem Manifest zur Methodik der *Critical Code Studies*. Dieser programmatische Aufruf zu einem philologischeren Umgang mit Code erschien – in einer früheren Version – bereits 2006 und konturiert nicht nur eine neue Methode an der Schnittstelle von Geisteswissenschaften und Computer Science. Vielmehr legt das Manifest ebenso systematisch wie einladend die Entwicklungsmöglichkeiten und Erkenntnisziele dar, wenn es darum geht, Code mit den Mitteln der literarischen Kritik, also philologisch zu lesen. Im Kern fordert dieser Ansatz dazu auf, Befehle auf ihre Verweisungszusammenhänge, auf ihre verborgenen Bedeutungen, auf das Spiel von Sprache und Metaphern, auf Ambiguitäten und Assoziationen, kurz, Codes auf ihre Rhetorizität und Poetik hin zu lesen.

Darauf aufbauend, zugleich aber auch abgrenzend und erweiternd verstehen wir die Tätigkeit der Code-Entwicklung in den Geisteswissenschaften – das Programmieren – als eine

¹⁴ Vgl. dazu sowie zur Ausweitung dieser Verfahren zu einem neuen Feld, den Critical AI Studies: Rita Raley und Jennifer Rhee, „Critical AI: A Field in Formation“, in: *American Literature* 95:2 (2023), S. 185–204.

genuine Kulturtechnik, die Verstehen vor allem durch Weiter-schreiben, durch Kommentieren und Einordnen erzielen kann. Der Beitrag von Markus Krajewski führt diesen Ansatz der Quellcodekritik aus der Perspektive der Kulturtechnikforschung anhand von drei Beispielen an verschiedenen Code-Fragmenten exemplarisch vor, um mit der Praktik des Kommentierens die epistemischen Vorzüge und Erkenntnispotentiale zu demonstrieren.

Till A. Heilmann stellt sich dem Problem der Unlesbarkeit von Code aufgrund seiner schieren Masse – selbst bei einem so alten Programm wie Photoshop 1.0.1. von 1990, das vom Hersteller Adobe als Code *a priori* öffentlich zur Einsicht freigegeben wurde. Die mehr als 100.000 Zeilen sind für Heilmann ein exemplarischer Fall für den Versuch, große Mengen von Code für Geistes- und Kulturwissenschaftler:innen ohne tiefere Programmierkenntnisse les- und analysierbar zu machen. Als Ansatz wählt er ein *distant reading* des Quelltexts, also eine oberflächliche, oft statistische Analyse mithilfe von Unix-Kommandozeilenwerkzeugen. Der Code wird schrittweise maschinell gelesen, transformiert und reduziert, um sein Panorama zu entfalten und erste Einsichten zu generieren. Dabei erweist er sich trotz seiner Funktionalität erneut als soziokulturell geprägt in seinen Sprachen, Paradigmen, Kommentaren und Benennungen.

Photoshop 1.0 konnte noch von einer einzigen Person – Thomas Knoll – geschrieben werden. Heute sind dagegen riesige Teams am Werk, deren Kommunikation untereinander meist über komplexe Protokolle und Dokumentationen geregelt ist. Aber nicht nur Menschen coden miteinander, sondern auch Maschinen mit Menschen. Dan Verständig untersucht in seinem Beitrag, wie Coding-Assistenzsysteme auf KI-Basis wie Codex und Copilot Programmieraufgaben teils automatisiert übernehmen und als *pair programmer* zusammen mit menschlichen Coder:innen agieren. Verständig betrachtet diese Pro-